DIGITAL DESIGN

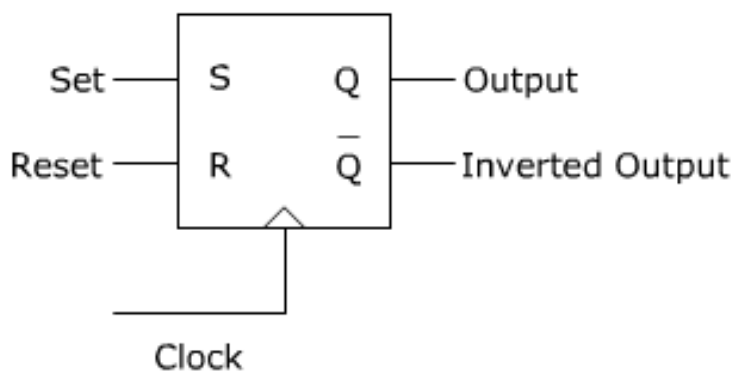
-MOHIT SINGH

FLIP FLOPS

- Flip-flops are the first stage in sequential logic design which incorporates memory (storage of previous states).
- Flip-flops that we will look at include the following:
  - SR type Flip-flop or Set / Reset
  - T type Flip-flop or Triggered /Toggle
  - D type Flip-flop or Data / Delay
  - JK type Flip-flop

SR Flip-flop (Set-Reset)

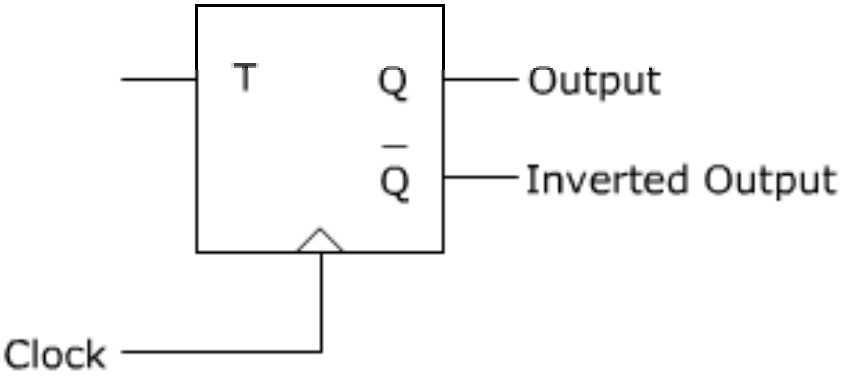
- Set input will make Q = 1 i.e. will 'set' the output.
- Reset input will make Q = 0 i.e. 'reset' the output.



Input		Circuit Action
S	R	$Q_{(Time\ t+1)}$
0	0	$Q_{(t)}$
1	0	1
0	1	0
1	1	Not Allowed

T flip-flop (Toggle/Triggered)

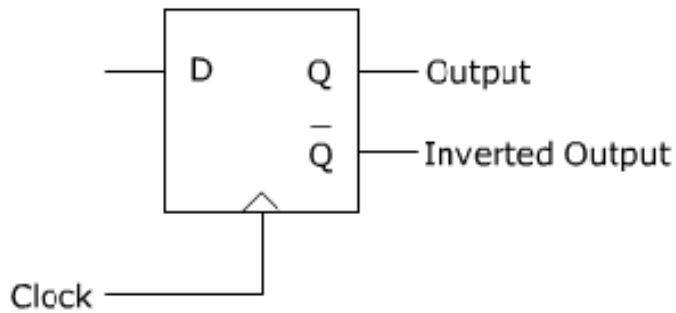
- A '0' input to 'T' will make the next state the same as the present state
- A '1' input to 'T' will change the next state to the inverse of the present state.



Input T	Circuit Action $Q_{(Time\ t+1)}$
0	$Q_{(t)}$
1	$Q_{(t)}'$

D flip-flop (Delay/Data)

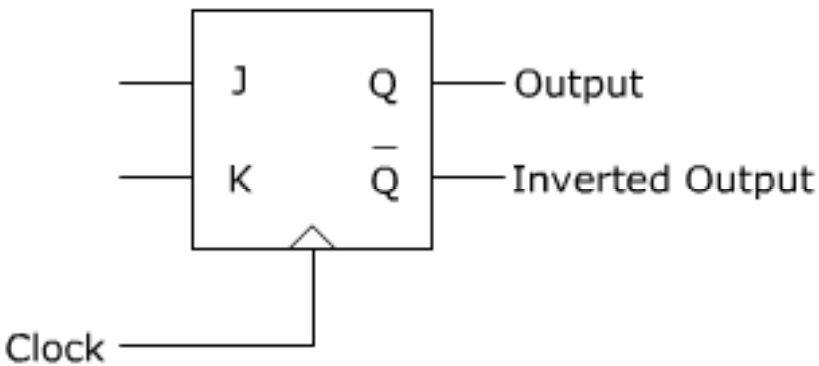
- Any input appearing (present state) at the input D, will be produced at the output Q in time T+1 (next state)



Input D	Circuit Action $Q_{(Time\ t+1)}$
0	0
1	1

JK flip-flop

- When J=K=0, Next state =Current state
- When J=0 and K=1, Next state= 0
- When J=1 and K=0, Next state =1
- When J=K=1, Next state =Current state Inverse



Input		Circuit Action
J	K	$Q_{(Time\ t+1)}$
0	0	$Q_{(t)}$
1	0	0
0	1	1
1	1	$Q_{(t)}'$

MAPPING (useful to make k-map for state diagram)

Present State	Next State	Inputs S	Inputs R
0	1	1	0
1	0	0	1
1	1	X	0
0	0	0	X

Present State	Next State	Input D
0	0	0
0	1	1
1	0	0
1	1	1

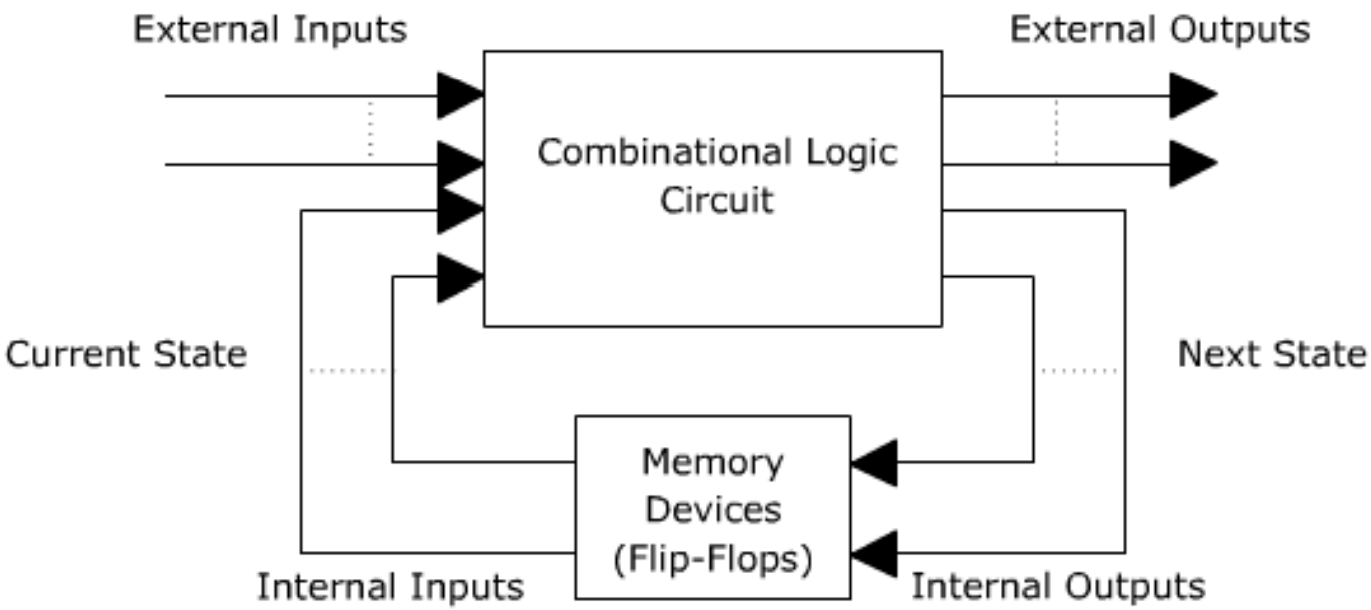
Present State	Next State	Input T
0	0	0
0	1	1
1	0	1
1	1	0

Present State	Next State	Inputs J	Inputs K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

SEQUENTIAL CIRCUITS

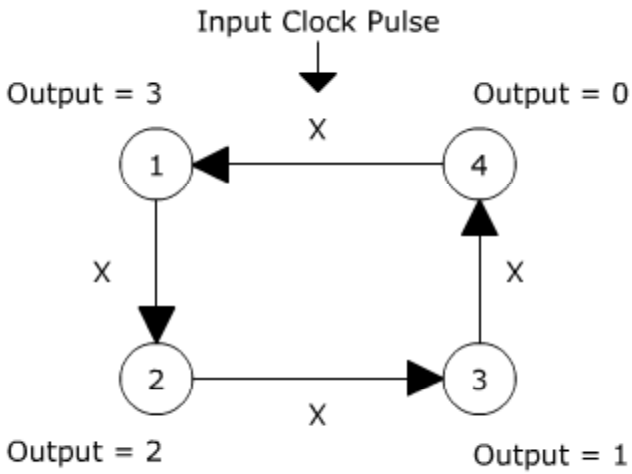
- **Combinational circuits** and systems produce an output based on input variables only.
- **Sequential circuits** use current input variables and previous input variables by storing the information and putting back into the circuit on the next clock (activation) cycle.
- There are two types of sequential circuit, **synchronous** and **asynchronous**.

SEQUENTIAL CIRCUITS



STATE DIAGRAMS & STATE TABLES

- To make a state diagram,
  - Define allowed states and label each state with an arbitrary number or letter along with a label showing the actual output
  - Define the transitions between related states and mark this on the diagram by arrows. Also mark near the line, the input that initiates the change



STATE TABLE

- The approach:
  - Select each of the allowed states in turn.
  - For each state write down the next state (note: could be multiple next states)
  - For each state note down the output level (or pulse) at the current state
  - Make state diagram
  -

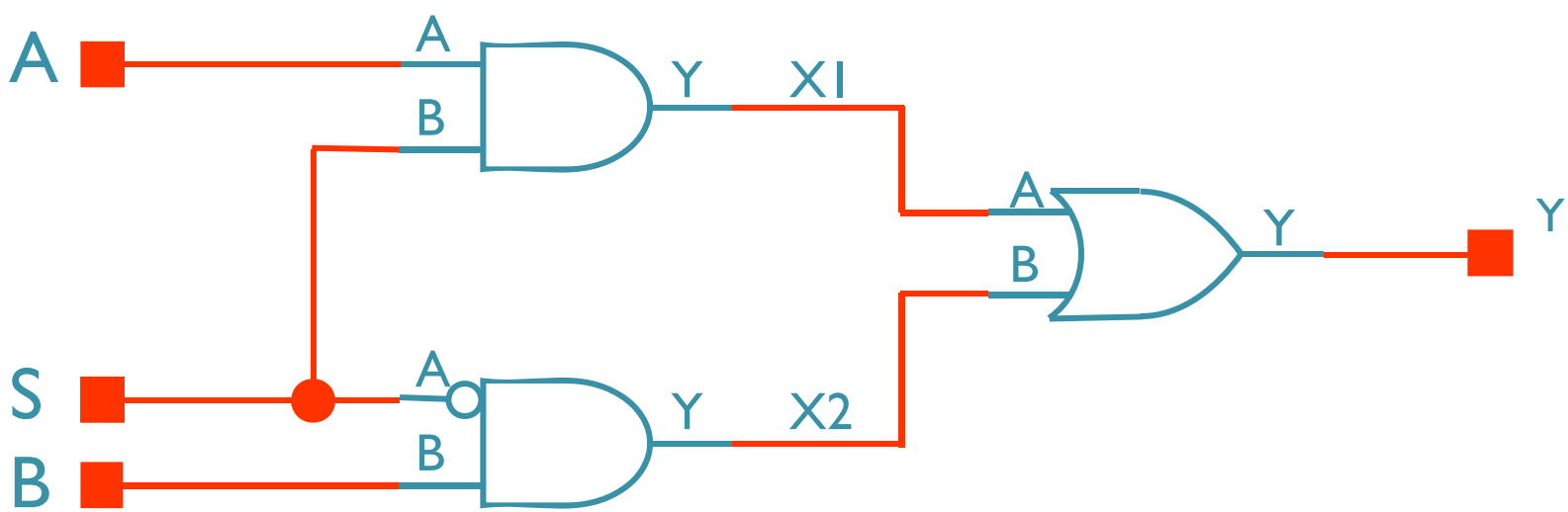
Present State	Next State	Output Z1, Z2
1	2	1, 1
2	3	1, 0
3	4	0, 1
4	1	0, 0

HAZARDS

- Hazards in any system are obviously an un-desirable effect caused by either a deficiency in the system or external influences. Mainly three types:
  - Static Hazards
  - Dynamic Hazards
  - Function Hazards
- When changes in the input variables do not change the output due to some form of delay caused by logic elements (NOT, AND, OR gates etc), this results in the logic not performing its function properly.
- ***Temporary problem***, logic finally comes to the desired function.
- Despite, it is imperative that hazards be eliminated as they can have an effect on other systems.

STATIC HAZARDS

- When one input variable changes, the output changes momentarily when it shouldn't.
- f = S.A + S'.B



MULTIPLEXER

STATIC HAZARDS

	A B			
	0 0	0 1	1 1	1 0
S=0	0	1	1	0
S=1	0	0	1	1

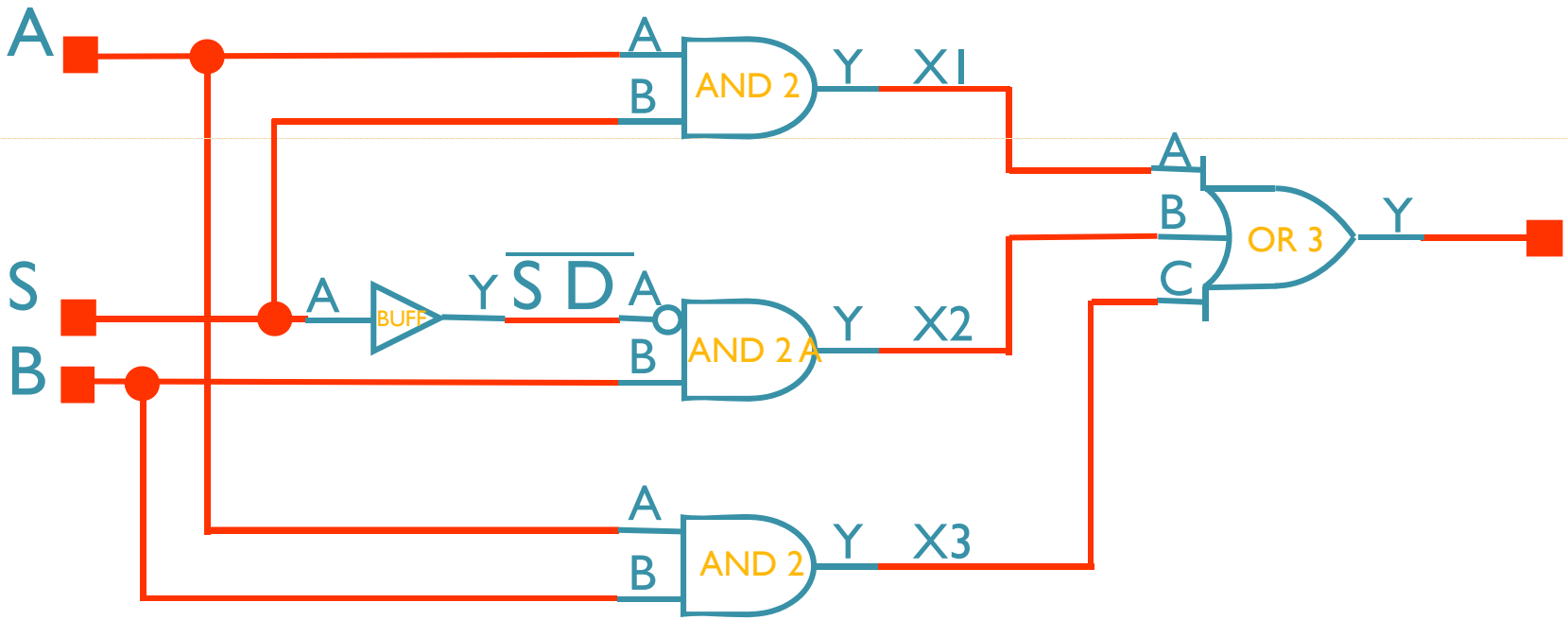
Illustrating the minimized function on a Karnaugh map.
Only two 2-input AND gates are needed for the product terms

STATIC HAZARDS

	A B			
	0 0	0 1	1 1	1 0
S=0	0	1	1	0
S=1	0	0	1	1

The blue oval shows the redundant term used to cover the transition between product terms.

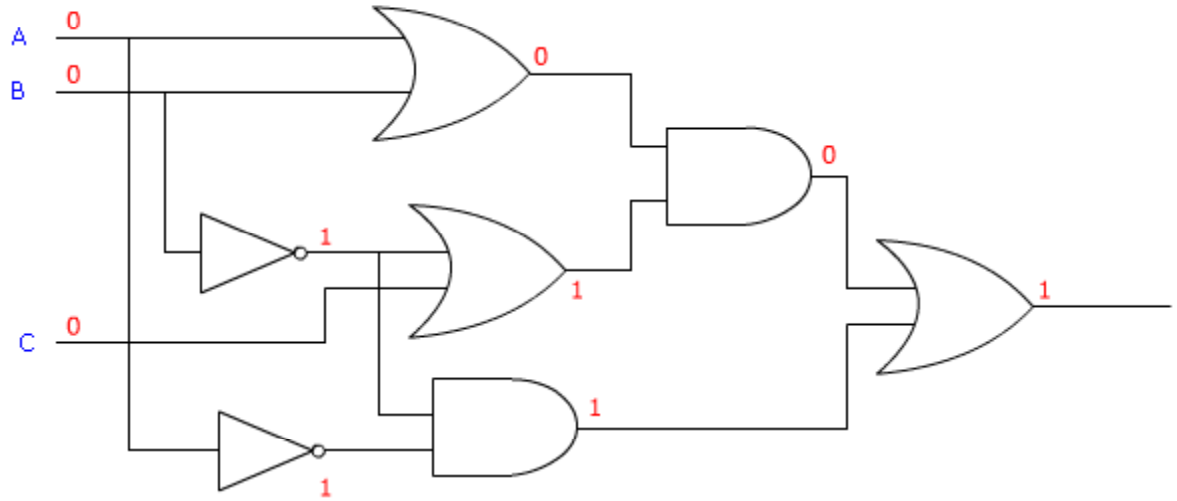
HAZARD FREE DESIGN



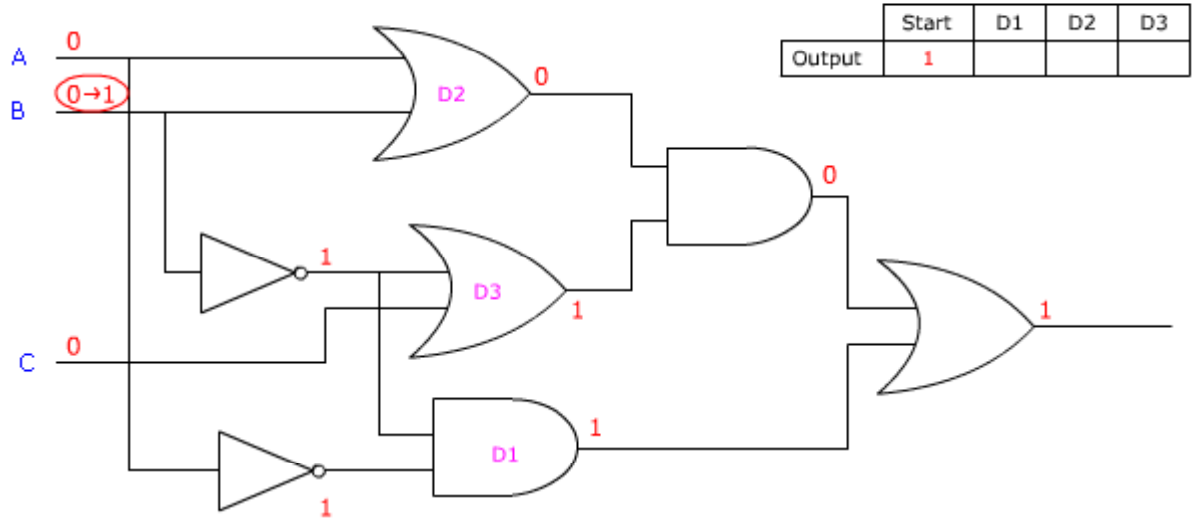
DYNAMIC HAZARDS

- A dynamic hazard is the possibility of an output changing more than once as a result of a single input change
- We will not cover how to fix them.

DYNAMIC HAZARDS



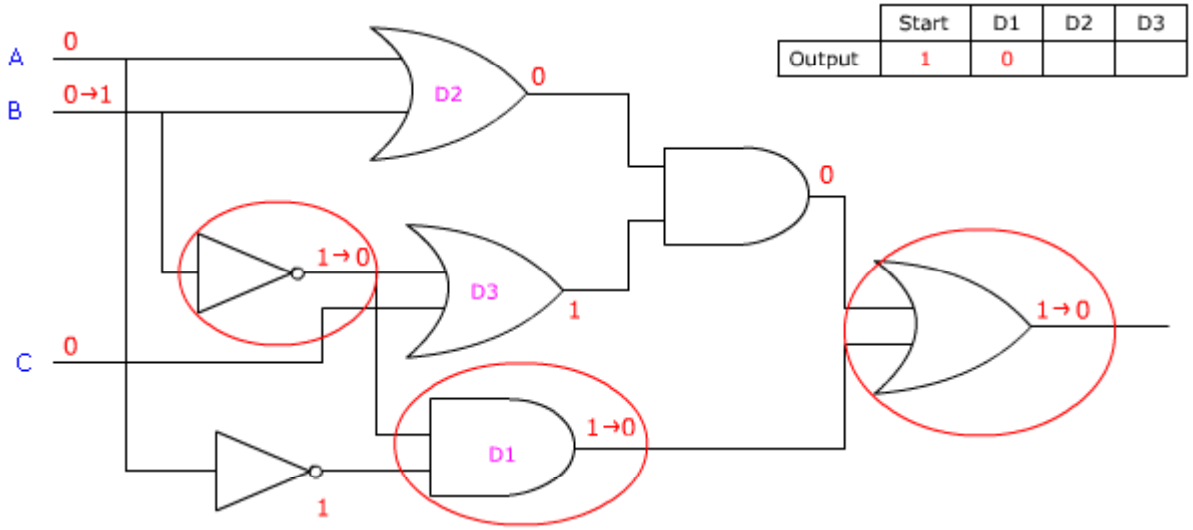
DYNAMIC HAZARDS



Logic values at each stage

	Start	D1	D2	D3
Output	1			

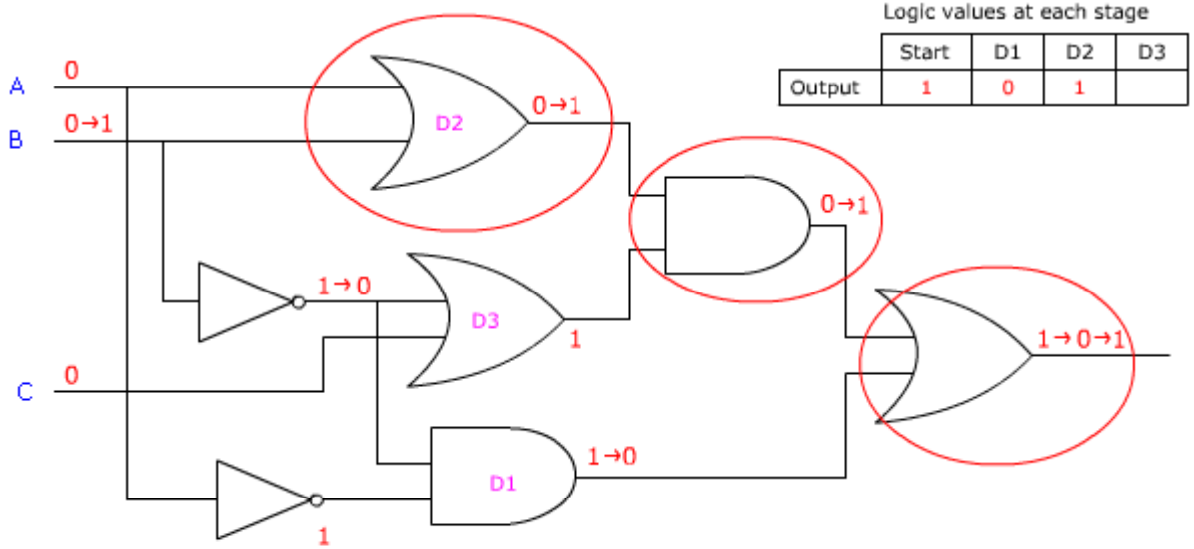
DYNAMIC HAZARDS



Logic values at each stage

	Start	D1	D2	D3
Output	1	0		

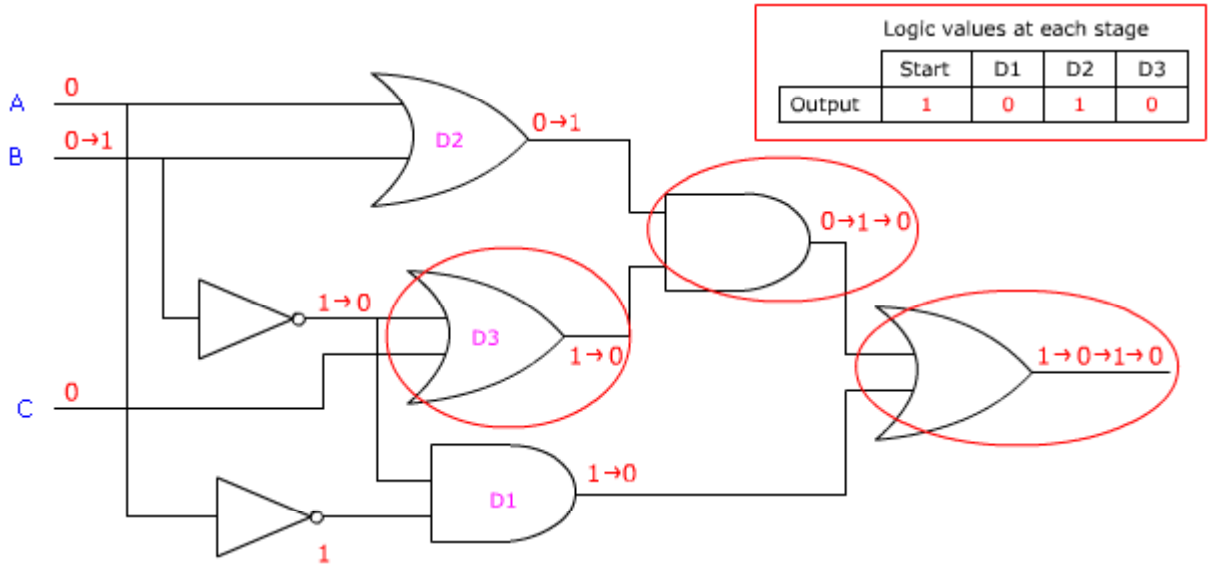
DYNAMIC HAZARDS



Logic values at each stage

	Start	D1	D2	D3
Output	1	0	1	

DYNAMIC HAZARDS



Logic values at each stage

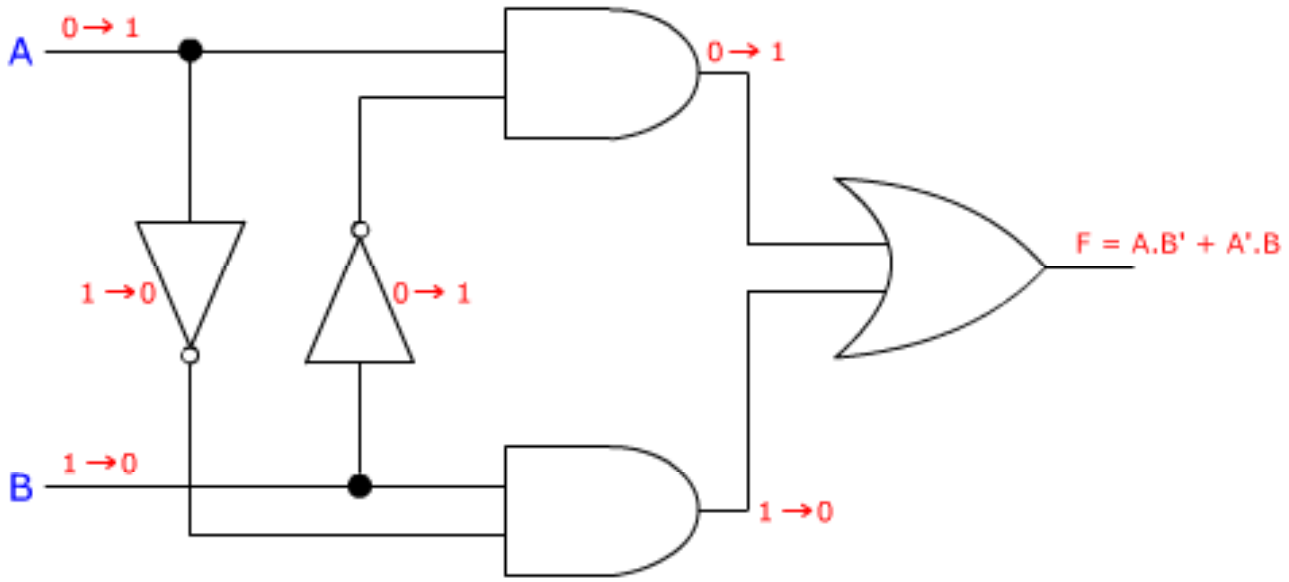
	Start	D1	D2	D3
Output	1	0	1	0

FUNCTIONAL HAZARDS

- Function hazards are non-solvable hazards which occurs when more than one input variable changes at the same time.
- Hazards such as function hazards can not be logically eliminated as the problem lies with actual specification of the circuit.
- The only real way **to avoid** such problems is to **restrict the changing of input variables so that only one input should change at any given time.**

FUNCTIONAL HAZARDS

EXAMPLE: XOR gate



FUNCTIONAL HAZARDS

B \ A	0	1
0		1
1	1	

- The most **effective way** to solve this hazard would be to **carefully design** the PCB so that delays are all equal, or at least match the delays on each path. i.e. Delay of A's path = Delay of B's path

Q. Using a 2:1 Mux realize:

- NOT gate
- AND gate
- OR gate
- Ex-OR gate
- Ex-NOR gate
- NAND gate
- NOR gate
- Latch
- FlipFlop

Use shanon's Expansion:

mux_{out}= sel' * Input0+ sel * Input1

Eg: AND gate Y=A * B

=A * B+A' * 0'

So, so we select ,

A as MUX control signal

Input0 is "0"

Input1 is "B"

Similarly for other gates.

REMEMBER:

1) XOR is not universal gate, only inverter and buffer can be made from it.

2) Similarly for conversion b/w FFs.

3) MUX is universal element in the sense, all gates can be made from it. Additionally memory element .

LATCH:

Connect Out to (Input0)

Input at Input1

Clock at Select line.

Q. Using combo logic make a multiply by two clock ($f_{out}=2*f_{in}$) (50% duty cycle)

- Use of a counter

Q. Using combo logic make a multiply by two clock ($f_{out}=2*f_{in}$) (50% duty cycle)

- Very systematic approach
  - Draw i/p & o/p waveforms
  - Delay the clock by T/4 and XOR

- Now try a multiply by 3 clock

Q. Using combo logic make a multiply by two clock ($f_{out}=2*f_{in}$) (50% duty cycle)

- Very systematic approach
  - Draw i/p & o/p waveforms
  - Delay the clock by T/4 and XOR

- Now try a multiply by 3 clock
  - Simple, delay by 1/3 & 2/3 and XOR three inputs.

Q. REGISTER SWAPPING

Q. SWAP REGISTERS A & B without use of any other register.

Q. SWAP REGISTERS A & B without use of any other register.

Ans: B = A XOR B

A = A XOR B

B = A XOR B

- How many 2:1 MUX you need to realize a 16:1 MUX?

- How many 2:1 MUX you need to realize a 16:1 MUX?
- 8 + 4 + 2 + 1 = 15

- What are the two ways to make an input inverter from a two input NAND gate?

- What are the two ways to make an input inverter from a two input NAND gate?

  1) Short the inputs

  2) Connect one terminal to logic 1.

- Realize a T-FF using 2:1 MUXes and few gates.

- Realize a T-FF using 2:1 MUXes and few gates.
- T-FF: Y= TQ' + T'Q

  MUX: Y = Sel'*A + S*B

  So, A=T, B = T'

  Use AND gate & inverter for edge-triggering

What is the difference between EEPROM & Flash memory?

- EEPROM requires data to be written or erased one byte at a time
- Flash memory allows data to be written or erased in blocks.
- This makes flash memory faster.

What is metastability?

- We think of flip-flops having only two stable states - but all have a third *metastable state* halfway between 0 and 1.
- When the setup and hold times of a flip-flop are not met, the flip-flop could be put into the metastable state.
- Noise will be amplified and push the flip-flop one way or other.

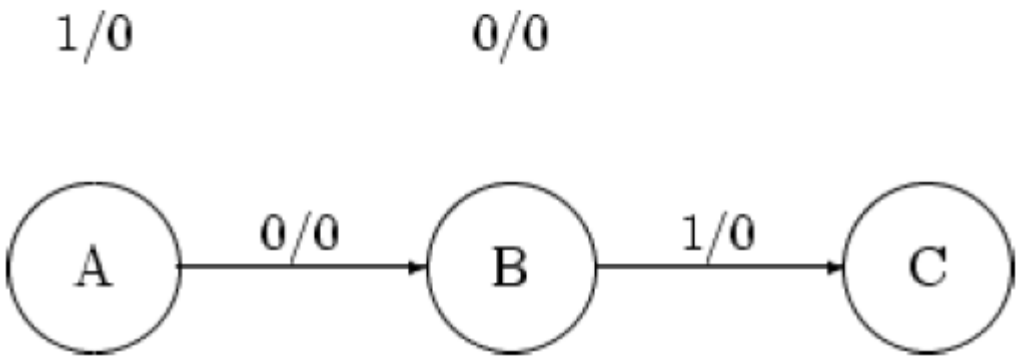
Why is NAND logic preferred in CMOS?

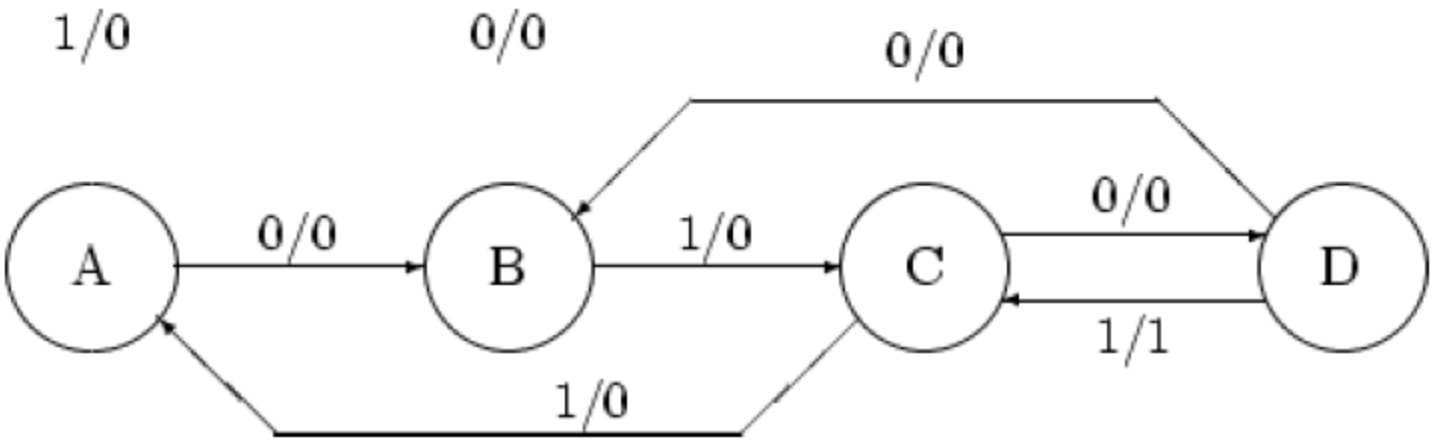
- NAND is better than NOR, because in a NOR gate we have PMOS transistors in series.
- When we enlarge transistor widths gate capacitance and diffusion capacitance go up and partially negate the benefits of widening the transistors so very wide PMOS transistors are super capacitive. Hence we prefer NANDs.

SEQUENCE DETECTOR (Approach)

- A state machine is required which outputs a logic 1 whenever the input sequence 0101 is detected, and which outputs a 0 otherwise.
- The input is supplied serially, one bit at a time. The following is an example input sequence and output sequence:

```
input   0 0 1 0 1 0 1 1 0 0 0 1 0 1 0 0
output  0 0 0 0 1 0 1 0 0 0 0 0 0 1 0 0
```


1/0
0/0
A
0/0
B
1/0
C


1/0
0/0
0/0
A
0/0
B
1/0
C
0/0
D
1/1
1/0

Present State	Input	Next State	Output
A	0	B	0
A	1	A	0
B	0	B	0
B	1	C	0
C	0	D	0
C	1	A	0
D	0	B	0
D	1	C	1

What is the difference between latch and flip flop?

- Misconception: Latch doesn't have a clock signal, but flip flop has.
- Latch is a level sensitive device while flip-flop is an edge sensitive device.
- Latch is sensitive to glitches at enable pin, whereas flip-flop is immune.
- Latches take less gates (also less power) to implement than flip-flops.
- Latches are faster than flip-flops.

What is the difference between glitch, jitter, skew?

- **Glitch**: In reality logic circuits have a non-zero propagation delay, finite propagation delay from one logic block to another causes invalid transitions called glitches.
- **Jitter:** Variations of clock period at a given point, that means clock period is not constant ,it may decrease or increase from cycle to cycle.
- **Skew**: This generally occurs due to path length mismatches. For example if the clock signal is given to a flip-flop circuitry containing 2 or more flip-flops, then if clock skew occurs they do not trigger simultaneously because of static path length mismatch[some delay comes. This mismatch is constant from cycle to cycle.

Digital Circuit

- What is the dependence of transition delay?
  $T(P_{HL})=1.7C/k_{(n)'}(W/L)_{(n)}V_{(VDD)}$
  $T(P_{LH})=1.7C/k_{(p)'}(W/L)_{(p)}V_{(VDD)}$

- what is the value of propagation delay?
  $T(P)=1/2*(T(P_{HL})+T(P_{HL}))$

- Equation for power consumption?
  P=fC*(V(DD))^2

- What is figure of merit in an inverter?
  Power*delay

FSM

- Finite State Machine: Tt is a model of behavior composed of a finite number of states, transitions between those states, and actions..
- There are mainly two type of Fsm

1)Moore machines

The o/p only depends upon different states.

Design is simple to implement

Has less control

E.g.: Freewheel Counter with controlled i/p

2) Mealy Machines

The o/p also depends upon the given i/p to the circuit with the help of combinational logic.

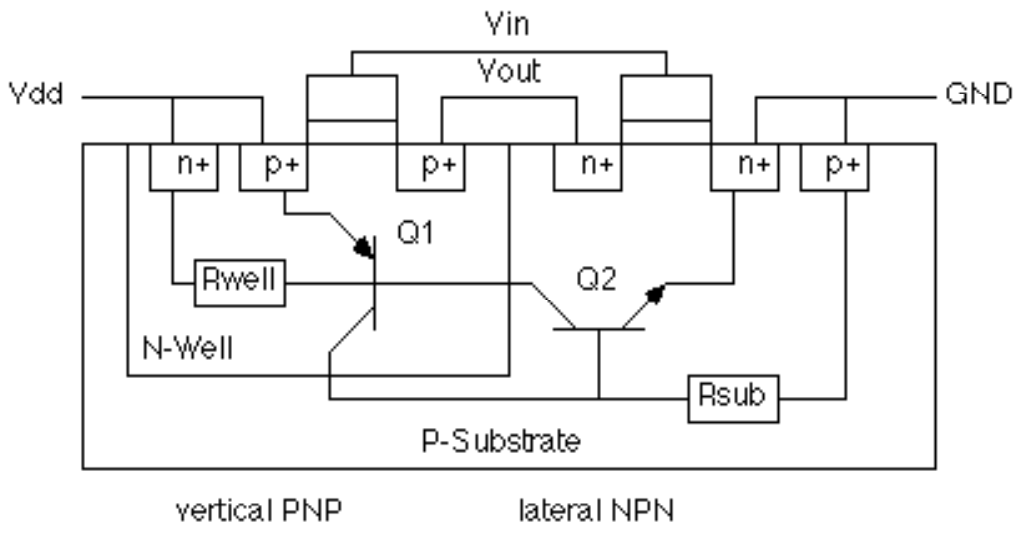
Design is complicated

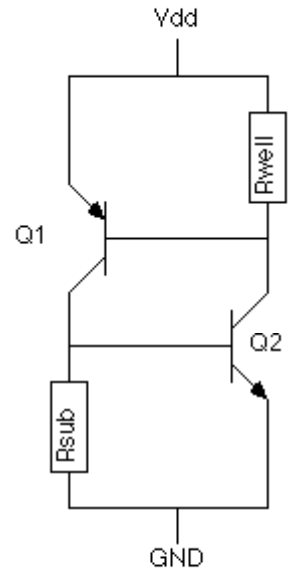
Have full control

E.g.: Counter with controlled i/p

4) What is CMOS Latchup?

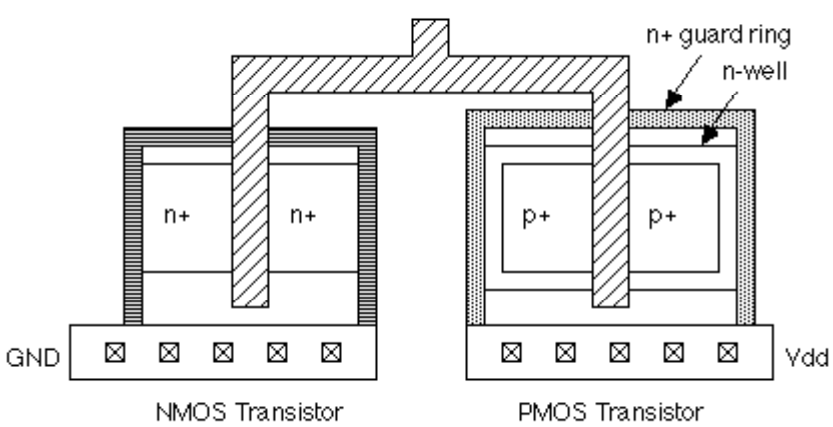
- Byproduct of the Bulk CMOS structure is a pair of parasitic bipolar transistors forming a positive feedback structure.
- The result of latchup is at the minimum a circuit malfunction, and in the worst case, the destruction of the device.





How to avoid CMOS Latchup

- Fab/Design Approaches
  - Reduce the gain product b1 x b1
    - move n-well and n+ source/drain farther apart increases width of the base of Q2 and reduces gain beta also reduces circuit density
    - buried n+ layer in well reduces gain of Q1
  - Reduce the well and substrate resistances, producing lower voltage drops
    - higher substrate doping level reduces Rsub
    - reduce Rwell by making low resistance contact to GND
    - guard rings around p- and/or n-well, with frequent contacts to the



- Systems Approaches
  - **Make sure power supplies are off before plugging a board**. A "hot plug in" of an unpowered circuit board or module may cause signal pins to see surge voltages greater than 0.7 V higher than Vdd, which rises more slowly to is peak value. When the chip comes up to full power, sections of it could be latched.
  - **Carefully protect electrostatic protection devices associated with I/O** pads with guard rings. Electrostatic discharge can trigger latchup. ESD enters the circuit through an I/O pad, where it is clamped to one of the rails by the ESD protection circuit. Devices in the protection circuit can inject minority carriers in the substrate or well, potentially triggering latchup.
  - **Radiation, including x-rays, cosmic, or alpha rays,** can generate electron-hole pairs as they penetrate the chip. These carriers can contribute to well or substrate currents.
  - **Sudden transients** on the power or ground bus, which may occur if large numbers of transistors switch simultaneously, can drive the circuit into latchup. Whether this is possible should be checked through simulation

What is electromigration?

- Transport of material caused by the gradual movement of the ions in a conductor due to the momentum transfer between conducting electrons and diffusing metal atoms.
- Important in applications where high direct current densities are used, such as in microelectronics and related structures.
- As the structure size in electronics such as integrated circuits (ICs) decreases, the practical significance of this effect increases.

How to avoid electromigration?

- **Wire metal:** Pure copper used for Cu-metallization is more electromigration-robust than aluminum (5 times).
- **Bamboo structure** and metal slotting
- **Blech length:** Lower limit for the length of the interconnect that will allow electromigration to occur.
- Via arrangements and corner bends
- Electromigration in Solder Joints: Electromigration occurs at much lower current densities
- **Doping metal** with heavy atoms.

What is ESD?

- Electrostatic discharge (ESD) is the sudden and momentary electric current that flows between two objects at different electrical potentials.
- Integrated circuits are made from insulating materials such as silicon dioxide can suffer permanent damage when subjected to high voltages. As a result there are now a number of antistatic devices that help prevent static build up.
- Q = C*V ;

  MOS devices C is very small

  =>large V even for small Q,

  =>oxide breakdown may occur.

What is Ground Bounce?

- usually seen on high density VLSI where insufficient precautions have been taken to supply a logic gate with a sufficiently low resistance connection (or sufficiently high capacitance) to ground.
- In this phenomenon, when the gate is turned on, enough current flows through the emitter-collector circuit that the silicon in the immediate vicinity of the emitter is pulled high, sometimes by several volts, thus raising the local ground, as perceived by the transistor, to a value significantly above true ground.
- Relative to this local ground, the BASE voltage can go negative, thus shutting off the transistor. As the excess local charge dissipates, the transistor turns back on, possibly causing a repeat of the phenomenon, sometimes up to a half-dozen bounces.
- Ground bounce is one of the leading causes of "hung" or metastable gates in modern digital circuit design. This happens because the ground bounce puts the input of a flip flop effectively at voltage level that is neither a one or a zero at clock time, or causes untoward effects in the clock itself.
- **A similar phenomenon may be seen on the collector side, called VCC sag, where VCC is pulled unnaturally low.**

How to avoid ground bounce?

- Flip Chip device packages
- Slow slew rate
- Limit the number of simultaneously switching output (SSOs) per I/O bank
- Spread out SSOs
- Programmable GND and VCC pins
- Out-of-phase simultaneously switching output (SSOs) using phase-locked loops (PLLs)
- Out-of-phase simultaneously switching output (SSOs) using programmable output delay option
- Series termination resistors
- Series-RC parallel terminations
- Decoupling capacitors

Why don't you use only an NMOS/PMOS in TG?

- For nMOS transistor, with OUT=high and IN=low and we are trying to transmit the low to the OUT, then the nMOS will drain some of the voltage but not all of it leaving the OUT somewhere in the 'no mans' voltage region of digital circuits.
- Adding the pMOS gate in parallel allows all the voltage to drain after the nMOS shuts off before all the voltage is drained.
- Similarly, when transmitting a high voltage to OUT.

What is self loading effect in CMOS?

- If you increase w/l more, then the op capacitance is also increased (due to drain depletion cap and miller cap.), this is self loading.
- This means a gate has to drive more to charge or discharge a large capacitor which is the result of making the gate a strong driver like see-saw.
- Moreover it offers more load to the next gate too.
- Sizing a gate is loaded by many guidelines taking account of all gates in the path, people call this as logical effort.

What is the diff b/w velocity saturation mode and saturation mode of operation of a mosfet?

- velocity saturation is pushing the electroncs to its maximum speed (say10^5 m/sec),beyond that it wont. Thereby the current=dq/dt no more increases even if u increase the V. V=Mobility*E/(1+E/Ec), where E = (V/d) and Ec = critical E(max).
- saturation mode on the other hand is due to Vds >=Vgs-Vt (pichoff).